Interface & Settings

Contents

- Grimoire Menu
  - Overview
  - Controls
  - Tabs
  - Items (Grimoire entries)
- Bestiary
  - Adding an enemy
  - Making a page
- Graphics Settings
  - What's on the page
  - How it works
  - Setup
  - Building the page
  - Adding a setting
- UI Render Target
  - Why
  - Setup
- Render Resolution Manager
  - Setup
- Gotchas

The game's menus and display settings:

- **Grimoire Menu:** the pause menu, with Inventory, Bestiary and Settings tabs.
- **Bestiary:** records kills and shows a page per enemy type.
- **Graphics Settings:** the Graphics page and the manager that applies it.
- **UI Render Target:** renders the HUD separately so it gets its own post-processing.
- **Render Resolution Manager:** the internal resolution setting.

The persistent managers here (Graphics Settings, Render Resolution) live in the **SceneEssentialsBundle** prefab.

See also: Shaders & VFX, Camera Effects, Fear Effects, World-space Labels & Interaction.

Grimoire Menu

Overview

The Grimoire is the book the player holds. Each page (**BookL** and **BookR**) has two sets of content:

- **Minimised UI** during gameplay: the current item and the scrollable item list.
- **Full Interface** when paused: the tabbed menu.

Opening the Full Interface pauses the game through the scene's `PauseManager` (time scale, cursor and input maps).

File	Job
`ALTGrimoire.cs`	Holds the player's items and opens, closes and switches the Full Interface.
`Panels/GrimoireInventoryPanel.cs`, `GrimoireBestiaryPanel.cs`, `GrimoireSettingsPanel.cs`	One script per tab.
`Panels/GrimoireDetailView.cs`	The right-page detail view (name, text, polaroid image).
`Panels/GrimoireEntryButton.cs`	A row in a tab's list.
`Panels/GrimoireTab.cs`	Tab enum: Inventory, Bestiary, Settings.

Controls

Input	Closed	Open
GrimoireUI (Tab / Q)	Opens to Inventory	Jumps to Inventory, or closes if already there
Pause (Escape / Start)	Opens to Settings	Jumps to Settings, or closes if already there
Shoulder buttons		Cycle tabs
X button, tab buttons		Close, or switch tab

The open keys are set in **Toggle Bindings** on `ALTGrimoire` (each is an input action plus the tab it opens).

The Grimoire won't open if something else (like dialogue) already has the game paused, or was closed on the same frame, so the Escape that closes dialogue doesn't also open the Grimoire.

> **In code:** the toggle actions are passed to `PauseManager.RegisterPersistentAction()` so they keep working while the Player input map is off. Opening switches every Animator under the

> Grimoire to unscaled time so the book animates while paused. `ALTGrimoire.instance.OnGrimoireToggled(bool)` fires on open and close. World labels and `GrimoireAnimManager` (the book's menu animation) listen to it.

Tabs

Each tab is a **panel**: an empty GameObject holding that tab's script. Only the active tab's panel is enabled. Panels build their content in `OnEnable` and clear it in `OnDisable`.

The Inventory and Bestiary tabs share the **ScrollView** on BookL and the **DetailView** on BookR (listed in **Shared List Layout**, hidden on the Settings tab).

Tab	What it does
Inventory	Lists the player's items. Selecting one makes it the held item and shows its details and image.
Bestiary	Lists killed enemy types. Selecting one shows that enemy's page prefab on BookR. See Bestiary.
Settings	The settings page on BookL. **Graphics** and **Debug** buttons open their pages on BookR (one at a time). **Quit** closes the Grimoire and loads **Menu Scene Build Index**.

When a tab opens, the first selectable under its **Tab Focus Container** is selected, so controllers can navigate without a mouse.

Adding a tab

1. Add a value to `GrimoireTab` and a heading to `tabNames` in `ALTGrimoire`.
2. Write a panel script that builds in `OnEnable` and clears in `OnDisable`.
3. Add its panel and tab button fields to `ALTGrimoire`, and add them to the `panels` and `tabButtons` arrays in `Awake` **in the same order as the enum**.
4. Hook up the button in `SetupButtons()`.

Items (Grimoire entries)

The player's items are `ALTGrimoireEntry`s in `ALTGrimoire.entries`. Each has a name, flavour text, hint text, an **Image** (shown in the polaroid) and a *collected* flag. Pickups add them through `CollectibleObject`.

> **In code:** `AddEntry(entry, collected)`, `CollectEntry(entry)`, `GetEntry(index or name)`, `GetCurrentEntry()` and `SelectEntry(index)`. `OnEntriesChanged` fires when an entry is added or collected. Entries are matched **by name**, and other systems rely on this (`AbilityManager` unlocks abilities by entry name).

Bestiary

An enemy type appears in the Bestiary the first time the player kills one, along with a kill count.

Adding an enemy

1. **Create > Game > Enemy Definition** and fill in:
    - **Id**: a stable id like `enemy_imp` (meant as a save key, don't change it later),
    - **Display Name**: shown in the list,
    - **Page Prefab**: the page shown on BookR.
2. Add **BestiaryTracker** to the enemy prefab and assign the definition.
3. Make sure a **Bestiary** component exists in the scene.

Making a page

Pages are UI prefabs, so each enemy's page can be laid out freely.

1. Create a UI prefab sized for BookR.
2. Lay out the heading, flavour text, picture and a TMP text for the kill count.
3. Add **BestiaryPage** to the root and assign **Kill Count Text**. **Kill Count Format** uses `{0}` for the number (default `Defeated: {0}`).

Pages are spawned under the Bestiary panel's **Page Parent**, keeping their own layout, so stretch Page Parent over the area pages should fill.

> **In code:** `BestiaryTracker` listens to `Enemy.OnDied` and calls `Bestiary.RecordKill(definition)`. The Bestiary raises `OnBestiaryChanged`, and the panel rebuilds. `HasDiscovered(definition)` and `GetRecord(definition)` are there for anything else that needs the data.

Graphics Settings

What's on the page

Shelf	Settings
Post-Processing	Disable All, Bloom, Chromatic Aberration (+ intensity), Downsampling pixel size, Colour Quantisation steps, Dithering (+ pattern), Sharpening (+ sample distance), CRT Blur / Scanlines / Vignette / Phosphor
VFX	Impact Frames, Fear Vignette intensity, Dash Effects intensity, the four Dash layers

Shelf	Settings
Camera Effects	Camera Shake, View Bobbing, Weapon Sway

Settings aren't saved. They reset to the designer's values each launch.

How it works

Post-processing settings go through a runtime **override Volume** that `GraphicsSettingsManager` creates on a dedicated layer, with a high priority. Every override starts **off**, so:

- an untouched setting shows the designer's value from the scene's Global Volume,
- turning an effect off overrides it to its "off" value,
- turning it back on removes the override, so the designer's value comes back,
- a slider or pattern overrides it with the player's value.

The designer can keep tuning the Global Volume, and the menu always starts from those values.

Other settings are properties that scripts read:

Property	Read by
`ImpactFramesEnabled`	`ImpactFrameController`
`CameraShakeEnabled`	`CameraEffects`
`ViewBobbingEnabled`, `WeaponSwayEnabled`	`WeaponSway`
`FearVignetteMultiplier` (0 to 1)	`FearPostProcessEffects`
`DashIntensityMultiplier` (0 to 1)	`PlayerDash`

Disable All turns off every Post-Processing shelf effect and greys out that shelf. The VFX and Camera Effects shelves aren't affected.

Setup

1. **GraphicsSettingsManager** lives in the SceneEssentialsBundle.
2. Create a **dedicated layer** for the override and enter it in **Volume Layer Name**.
3. Add that layer to the **Volume Mask** of **PlayerCam** and the **UI camera**.
4. Set **Designer Volume Mask** to the Global Volume's layer only (never the override layer).
5. Keep **Volume Priority** above every scene Volume.

Building the page

- **GraphicsShelf**: a header button that expands and collapses a **Content** object, with an optional rotating **Arrow**. Several can be open at once.

- **GraphicsToggleRow** / **GraphicsSliderRow**: pick the **Setting** from the dropdown and they wire themselves up. Ranges and value labels come from the manager.
- **GraphicsDitherPatternRow**: one Toggle per pattern, in order 2×2, 4×4, 8×8, 16×16, PS1.

Rows grey out (with an optional **Row Group** CanvasGroup) when they don't apply, for example the intensity slider while its toggle is off.

For a "toggle plus slider" setting, use a Toggle Row and a Slider Row together.

Adding a setting

A post-processing setting:

1. Add a value to `GraphicsToggle` or `GraphicsSlider`.
2. If its Volume Component isn't in the override profile, add it in `CreateOverrideVolume()`.
3. Add a nullable field for the player's value and read the designer's value in `RefreshDesignerValues()`.
4. Handle it in the matching `Get` / `Set` methods and in `ApplyOverrides()`.
5. Sliders: add a range in `GetSliderRange()` and a label format in `FormatSlider()` if needed.
6. Add a row to the page.

A script setting: add the enum value and a public property, handle it in `GetToggle` / `SetToggle`, and read it from the script. Treat a missing manager (`GraphicsSettingsManager.Instance == null`) as "on" so scenes without it still work.

> **In code:** rows listen to `OnSettingsChanged`, which fires after every change. Any script that searches for a Volume (to find its own effect) should skip `GraphicsSettingsManager.Instance.OverrideVolume`.

UI Render Target

Why

The HUD needs its own post-processing pass. On an overlay canvas it would get none, and in PlayerCam's camera stack the world's effects would be applied to it a second time. Instead the HUD renders on its **own camera** into a **transparent texture**, which is then drawn over the finished frame.

Setup

1. Make the UI camera a **Base** camera (not in PlayerCam's stack), rendering only the **UI** layer, with **Post Processing** on and the UI Volume's layer in its Volume Mask.

2. **Disable the camera's Camera component** in the scene.
3. Add **UIRenderTarget** to it.
4. Make a Screen Space - Overlay canvas with a full-screen **RawImage** using the **UI/PremultipliedComposite** shader. **Disable the RawImage** in the scene.
5. Assign it to **Composite Image**.

`UIRenderTarget` creates the texture at the world's internal resolution, points the camera at it, then turns the camera and RawImage on. It remakes the texture whenever the window size or render scale changes.

> **In code:** the texture is HDR with alpha (`R16G16B16A16_SFloat`) and a depth-stencil buffer so UI Masks work, with Point filtering to match the world. The composite shader blends with `One, OneMinusSrcAlpha` because the UI's colours are already multiplied by alpha. The default UI shader would darken soft edges.

The Downsample effect skips any camera named **UICam**, so the HUD isn't pixelated.

Render Resolution Manager

The player picks an **internal render resolution** (1080p, 1440p or 4K) in the Settings page's dropdown. The window resolution isn't changed. The world renders at the chosen height and is scaled to fit the screen (higher than the screen = supersampling).

It also sets a global shader value `_ResolutionScale` (chosen height / 1080). Post-process effects are authored at 1080p and use it to look the same at any resolution. See Shaders & VFX.

Setup

- **RenderResolutionManager** lives in the SceneEssentialsBundle.
- **ResolutionPicker** goes on the Settings page's TMP_Dropdown. Resolutions bigger than the display are marked with an asterisk.

To offer more resolutions, add them to `targetResolutions` in the manager.

> **In code:** the manager sets URP's `renderScale` to chosen height / screen height, clamped to 0.1 to 2, and recalculates when the window resizes. It restores the asset's original render scale when destroyed so Play Mode doesn't leave the pipeline asset changed. `SetResolution(width, height)` and `OnResolutionChanged` are available to other scripts.

Gotchas

- **Settings reach the world but not the HUD (or the other way round)?** Check the override layer is in both cameras' Volume Masks.
- **Never put the override layer in Designer Volume Mask.**
- **Only one camera per image should have Post Processing on**, or effects apply twice.
- **Leave the UI camera and composite RawImage disabled in the scene.**
- **Grimoire entry names must be unique.**
- **Panel order must match the `GrimoireTab` enum.**
- **An EnemyDefinition with no Display Name shows as a blank Bestiary row.**